

Log Analysis System Based on Large Language Models

Xueliang Gao^{1,2}

¹ School of Network and Information Security, Xidian University

² National Computer Network Intrusion Protection Center, University of Chinese Academy of Sciences

Xian, China

24151213621@stu.xidian.edu.cn

Shangru Zhao

School of Computer Science and Technology

University of Chinese Academy of Sciences

Beijing, China

zhaosr@nipc.org.cn

Mo Pang

Security Research Institute
China Academy of Information and Communications Technology

Beijing, China

pangmo@caict.ac.cn

Haoxing Zhang

Security Research Institute
China Academy of Information and Communications Technology

Beijing, China

zhanghaoxing@caict.ac.cn

Yuqing Zhang*

National Computer Network Intrusion Protection Center

University of Chinese Academy of Sciences

Beijing, China

zhangyq@ucas.ac.cn

Abstract—Logs record the running status of the system and important information. Developers can analyze logs to understand the system performance and detect anomalies. Therefore, log analysis plays a crucial role in ensuring the stability of software. However, with the increasing complexity of modern software systems, traditional log analysis methods are difficult to handle the heavy log analysis tasks. To address this challenge, this paper designs a log analysis system with large language models (LLMs) as its core. The system automatically collects log data using logstash and performs in-depth analysis by leveraging the powerful natural language understanding capabilities of LLMs. The implementation follows three steps: first, introduce the research status in the field of log analysis and related knowledge; second, describe the design of the system from the overall architecture to the specific modules; and finally, evaluate the system. Through modular design, the system achieves high efficiency and scalability in log data processing and can meet real-time system demands.

Keywords—log analysis, large language models, log collection

I. INTRODUCTION

With the rapid development of computer technology, the scale and complexity of modern software systems are showing a superlinear growth trend. This evolution, while significantly enhancing system functionality and performance, has also introduced new potential risks and challenges. In a complex system environment, even a minor fault may trigger a cascading chain reaction, ultimately leading to widespread system failures or crashes. For instance, an undetected memory leak in a critical service may gradually consume system resources, eventually causing the node to crash unexpectedly. Similarly, network latency or database bottlenecks can propagate across distributed systems, resulting in performance degradation or even complete outages.

Conventional monitoring metrics often fail to capture the root causes of such subtle yet critical issues. This is where the value of system logs becomes evident. Logs provide a detailed, time-stamped record of system behavior, including error messages, warnings, and execution traces, enabling engineers to perform in-depth root cause analysis. By leveraging

advanced log analysis techniques, teams can proactively identify and mitigate risks before they escalate into major incidents. Thus, in today's increasingly complex IT landscapes, logs serve as an indispensable tool for ensuring system stability and reliability.

As the core data carrier for recording the running status of a system, logs play a crucial role in software systems. They provide detailed insights into application behavior, error conditions, and performance bottlenecks, serving as a vital resource for troubleshooting and optimization. Developers can quickly detect system anomalies through log analysis, enabling efficient fault location and recovery. However, effective log analysis also faces significant challenges [1]. First, inconsistent log formats across different services or components make automated parsing difficult. Second, modern distributed systems generate an overwhelming volume of logs, complicating storage and real-time processing. Additionally, logs often contain a large amount of noisy data which obscures critical information.

Traditional log analysis primarily relies on rule-based approaches, with the open-source ELK Stack being a typical representative system. The ELK Stack consists of three core components: Logstash is responsible for log collection and preprocessing, Elasticsearch provides distributed storage and retrieval capabilities, and Kibana offers visualization functionality, enabling real-time processing of petabyte-scale data [2].

However, this rule-based analysis model exhibits significant limitations when dealing with new log formats. Since different systems and applications generate logs in various formats, administrators need to manually define and maintain a large number of parsing rules. This not only increases operational complexity but also leads to higher maintenance costs. Moreover, when systems are upgraded or new components are introduced, the existing rule base often fails to adapt quickly, resulting in parsing failures or inaccurate analysis results.

In recent years, machine learning and deep learning technologies have revolutionized the field of log analysis, offering innovative approaches to detect anomalies, identify

* Corresponding Author

patterns, and enhance system security. Traditional log analysis methods often rely on rule-based systems or manual inspection, which are time-consuming, error-prone, and unable to handle the vast volumes of data generated by modern IT infrastructures. In contrast, machine learning and deep learning techniques provide automated, scalable, and adaptive solutions that can uncover hidden insights from logs with minimal human intervention.

One notable advancement in this domain is VLAS [3], a versatile log analysis system that integrates seven distinct machine learning algorithms. By supporting flexible algorithm selection, VLAS allows users to tailor anomaly detection methods to their specific needs, improving accuracy and efficiency. MMSLAS [4] takes a holistic approach by analyzing multiple types of security logs simultaneously, leveraging correlations between different log sources to enhance detection capabilities. This method is particularly valuable in complex IT environments where security threats may manifest across various subsystems.

Commercial solutions like Splunk [5] further demonstrate the power of combining structured query languages with machine learning. Splunk's Search Processing Language enables users to perform sophisticated log searches, while its built-in machine learning modules facilitate predictive analytics, clustering, and anomaly detection. This integration makes Splunk a powerful tool for enterprise-level log analysis, helping organizations detect security breaches, optimize performance, and ensure compliance.

DeepLog [6] has also shown remarkable success in log anomaly detection. DeepLog employs Long Short-Term Memory (LSTM) networks [7] to model sequential log patterns, enabling it to detect deviations that may indicate system failures or security incidents. Due to its ability to capture long-term dependencies in log sequences, DeepLog achieves high detection accuracy and has been widely adopted in both research and industry.

However, despite these advancements, significant challenges remain. Most machine learning and deep learning-based log analysis methods require large amounts of labeled training data, which can be costly and labor-intensive to obtain. Additionally, these models may struggle to adapt to dynamic environments where log formats and system behaviors frequently change. Future research should focus on developing semi-supervised or unsupervised learning techniques to reduce reliance on labeled data, as well as methods for continual learning to improve adaptability in evolving systems. By addressing these limitations, next-generation log analysis systems can become even more robust and widely applicable.

The log analysis solution based on large language models (LLMs) proposed in this study effectively overcomes the limitations of traditional machine learning and deep learning approaches. Unlike conventional methods that require extensive labeled data and struggle with dynamic log formats, this solution leverages the advanced capabilities of LLMs to achieve more flexible, efficient, and cost-effective log analysis.

The proposed framework offers three key advantages: First, it employs a pre-training and fine-tuning mechanism, enabling the model to adaptively parse diverse log formats without manual feature engineering. This significantly improves generalization across different systems and reduces

dependency on structured log templates. Second, thanks to the deep semantic understanding of LLMs, the solution can detect complex abnormal patterns that traditional rule-based or statistical methods might miss. By analyzing contextual relationships and subtle anomalies in log sequences, it enhances detection accuracy while reducing false positives. Finally, the system reduces operational costs by utilizing open-source LLMs that can be deployed locally, eliminating the need for expensive cloud-based AI services or proprietary software. By combining the strengths of LLMs with log analysis, this approach provides a new direction for next-generation log analysis technology, paving the way for more adaptive and scalable intelligent operation and maintenance solutions.

II. RELATED KNOWLEDGE

A. Overview of Log

Logs are structured or unstructured text files that systematically record the runtime status, operational events, and critical information of software systems. Serving as an indispensable diagnostic tool, they play a vital role throughout the entire software lifecycle - from development and testing to deployment and maintenance. By providing detailed chronological traces of system behavior, logs enable developers and operations teams to quickly identify anomalies, diagnose root causes, and resolve issues, significantly improving debugging efficiency and enhancing overall system reliability.

A typical log entry consists of three fundamental components: header messages, static templates, and dynamic variables. Header messages contain metadata like timestamps and severity levels. Static templates are fixed text structures that describe event types. Dynamic variables are runtime-specific values that populate the templates. These components work together to create meaningful records of system activities while maintaining a balance between human readability and machine parsability.

However, significant challenges arise from the substantial variations in log formats across different systems and platforms. These format discrepancies create obstacles for automated log analysis tools and require flexible parsing approaches to handle effectively. The heterogeneity extends beyond operating systems to include differences between various applications, middleware, and cloud services, making unified log processing a complex but crucial task in modern IT environments. Fig. 1 shows the logs of the Windows system and the Linux system.

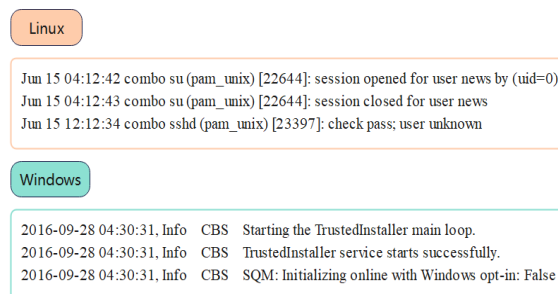


Fig. 1. Log messages.

Logs serve as critical diagnostic tools in computing environments, and their classification according to content

differences enables more targeted and effective analysis. The three primary log categories each fulfill distinct yet complementary roles in system monitoring and maintenance.

Application logs represent the most granular category, meticulously recording program execution events, transaction details, and error messages. These logs are indispensable for developers during debugging processes, as they provide visibility into application flow and variable states. Beyond troubleshooting, application logs facilitate performance optimization by tracking execution times, resource consumption, and throughput metrics. Modern applications often generate logs at multiple severity levels to support tiered analysis approaches.

System logs operate at a lower level, documenting the operational status of hardware components, operating system kernels, and core services. These include boot sequences, device driver activities, memory allocation events, and system call traces. System administrators rely on these logs for capacity planning, hardware fault diagnosis, and OS configuration troubleshooting. The format and collection methods vary significantly across platforms, with Windows using Event Logs, Linux employing syslog, and network devices typically generating SNMP traps.

Security logs form the foundation of organizational cybersecurity postures, recording authentication attempts, access control changes, firewall activities, and intrusion detection alerts. These logs are particularly valuable for forensic investigations following security incidents, as they provide chronological evidence of compromise vectors and attacker movements. Regulatory frameworks like PCI-DSS and HIPAA mandate specific security logging requirements, making these logs essential for compliance audits. Advanced Security Information and Event Management systems often aggregate and correlate security logs across multiple systems to detect sophisticated threats.

This tripartite classification system enables organizations to implement appropriate collection, retention, and analysis strategies for each log type. While application logs might prioritize immediate availability for developers, security logs typically require longer retention with strict access controls. Understanding these categorical differences allows for optimized storage solutions, analysis tool selection, and operational workflows across IT departments. The classification also informs log normalization efforts, as each category presents unique challenges in parsing and interpretation due to varying structure, verbosity, and terminology standards.

B. Overview of Log Analysis

Log analysis is a critical process in IT operations and security management, involving the systematic collection, processing, and interpretation of log data generated by computer systems, network devices, and applications. By leveraging automated technologies, organizations can extract valuable insights from vast amounts of log data, enabling them to detect anomalies, troubleshoot issues, and make informed decisions. This process is particularly important in modern IT environments, where the complexity and scale of systems make manual log inspection impractical.

The log analysis pipeline typically consists of three key stages: collection, preprocessing, and analysis. During the collection phase, logs are aggregated from various sources,

such as servers, firewalls, and applications, often using tools like Fluentd, Logstash, or specialized agents. The preprocessing stage involves cleaning and normalizing the raw log data, which may include filtering irrelevant entries, parsing unstructured text, and standardizing formats. This step is crucial for ensuring data quality and consistency.

The analysis phase addresses the challenge of log diversity by transforming raw text into structured data. Techniques such as pattern recognition, rule-based correlation, and machine learning are employed to identify meaningful patterns and anomalies. Predefined rules can detect known issues, while advanced deep learning models can uncover hidden trends or predict potential failures.

Log analysis technologies have evolved from rule-driven to data-driven. Early systems relied on predefined rule bases or signature matching to detect known error patterns, but they had difficulty dealing with complex and changeable log contexts. Statistical methods identify outliers by quantifying the frequency distribution or similarity of log events, but the cost of feature engineering is relatively high. With the development of deep learning, LSTMs and Transformer models have been used to model the temporal dependencies of log sequences. For example, anomaly detection is achieved through the prediction of log key sequences.

C. Overview of LLMs

LLMs represent a groundbreaking advancement in artificial intelligence, constructed through self-supervised training on massive text corpora. These models have demonstrated exceptional multilingual processing capabilities, significantly enhancing both the efficiency and quality of natural language processing (NLP) tasks. From machine translation to text summarization, and from sentiment analysis to question answering systems, LLMs have revolutionized how machines understand and generate human language.

The technological evolution of LLMs is rooted in decades of progress in deep learning and NLP. Early language models primarily relied on Recurrent Neural Networks (RNNs) [8], which process sequential data through internal recurrent connections to model temporal dependencies. While RNNs achieved some success in handling sequential data, they suffered from critical limitations, most notably the vanishing or exploding gradient problem. These issues made it challenging for RNNs to capture long-range dependencies in text, severely restricting their effectiveness in understanding complex linguistic structures.

To address these shortcomings, researchers developed more advanced architectures, including LSTMs and Gated Recurrent Units (GRUs). These variants introduced gating mechanisms to better control information flow, partially mitigating the gradient problems. However, they still struggled with parallelization and efficiency when processing large-scale data.

The introduction of the Transformer architecture marked a paradigm shift in NLP. Unlike RNNs and their variants, Transformers rely entirely on attention mechanisms, eliminating the need for recurrent or convolutional structures. This innovation addressed two major limitations of previous models: the inability to handle long-range dependencies efficiently and the lack of parallel processing capabilities.

The Transformer architecture consists of two main components: an encoder and a decoder. The encoder processes

the input sequence and converts it into a rich hidden state representation, while the decoder uses this representation to generate the output sequence step-by-step. The key innovation lies in the self-attention mechanism, which allows the model to weigh the importance of different words in a sequence dynamically. This enables the model to capture relationships between words regardless of their positional distance, solving the long-range dependency problem that plagued earlier architectures.

D. Overview of Fine-tuning

Fine-tuning is a critical process in adapting pre-trained LLMs to specific tasks or domains. During pre-training, LLMs acquire broad linguistic knowledge from vast amounts of general text data, but they may not be optimized for specialized applications. Fine-tuning addresses this gap by further training the model on task-specific datasets, enabling it to better understand the nuances, semantics, and contextual requirements of the target domain. This process not only enhances the model's performance in specialized tasks but also helps mitigate overfitting while ensuring greater stability and generalization in real-world applications.

The success of fine-tuning heavily depends on the quality of the dataset used. A well-curated dataset that accurately represents the target task can significantly improve the model's precision and adaptability. Conversely, a noisy or biased dataset may lead to suboptimal performance or even degrade the model's capabilities. Therefore, data selection, cleaning, and augmentation are essential steps in preparing an effective fine-tuning dataset.

Low-Rank Adaptation (LoRA) [9] is an efficient fine-tuning technique for LLMs. It employs a low-rank decomposition strategy to introduce small trainable adaptation layers beside the original parameters. Compared with traditional methods, LoRA has four major advantages:

- LoRA lowers the computational overhead required for fine-tuning LLMs. Instead of updating all parameters, LoRA introduces lightweight, low-rank matrices that modify only a small fraction of the model's weights. This approach drastically reduces memory usage and training time while maintaining model performance.
- Unlike traditional fine-tuning, LoRA keeps the original pre-trained model parameters frozen. By only training LoRA layers, the same base model can support multiple downstream tasks simply by switching different LoRA modules. This modular design enhances efficiency, as a single foundational model can serve various applications without redundant copies.
- Despite its parameter efficiency, LoRA achieves performance on par with full-parameter fine-tuning in most NLP tasks. The low-rank decomposition effectively captures task-specific adaptations without compromising the model's generalization capabilities. This makes LoRA a cost-effective solution for adapting large-scale models like GPT, Llama, and BERT [10] with minimal resource expenditure.
- LoRA's lightweight nature makes it highly scalable for real-world applications, enabling efficient updates and fine-tuning even on moderate-grade hardware. Its compatibility with existing model architectures further

simplifies adoption, making it a preferred method for resource-efficient model adaptation.

III. DESIGN OF LOG ANALYSIS SYSTEM

Given the scale of modern software systems, a short-term crash will cause incalculable losses. Therefore, log analysis has become an important part of maintaining system reliability. Log analysis system should possess two core capabilities: a generalized parsing function that supports multiple log formats, and an intuitive result visualization function.

The log analysis system based on LLMs uses Ubuntu as the operating system, Vscode as the development tool, and implements the entire system in a Python environment.

A. System Architecture

The log analysis system based on LLMs mainly consists of four functional modules: log collection module, log analysis module, data storage module, and data visualization module. Fig. 2 depicts the system architecture diagram.

- Log collection module is a core component responsible for systematically gathering raw log data from diverse sources, including servers, applications, and network devices. It supports multiple collection methods to ensure comprehensive data acquisition. Once collected, the module performs initial preprocessing tasks to improve data quality.
- Log analysis module is responsible for receiving the collected log data and classifying and grading the data according to predefined rules. By using the API interface to call the locally deployed LLMs, this module can conduct in-depth analysis of the log data, identify potential problems and provide corresponding analysis results.
- Data storage module receives the results processed by log analysis module and stores the analyzed data in the local MySQL database. By using a relational database, it can ensure the efficient storage, query, and management of log data.
- Data visualization module connects to the data stored in the database and presents the analysis results to users in the form of charts. The data visualization tool will assist users in more intuitively understanding the changing trends of log data.

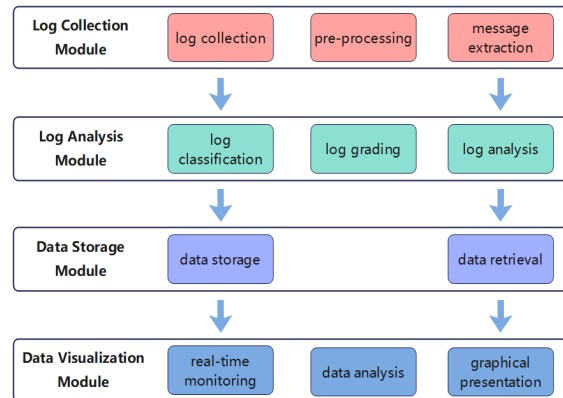


Fig. 2. The system architecture diagram.

B. Design of Log Collection Module

Log collection module is a crucial component of the log analysis system. It is mainly responsible for efficiently collecting raw log data from various data sources and performing preprocessing. This module uses Logstash as the core tool, which can connect to different data sources through its multiple input plugins to obtain log data in real-time. Logstash can flexibly specify log sources and collection parameters through configuration files, ensuring that it can adapt to different types of log sources and guarantee efficient log collection. Fig. 3 illustrates the log collection workflow of Logstash.

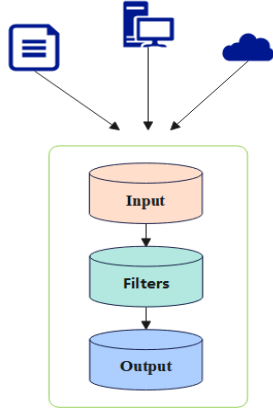


Fig. 3. The workflow of Logstash.

The collected logs are preprocessed according to Grok rules. The Grok plugin can extract key information such as timestamps, log levels, event types, and error messages from the logs based on predefined regular expressions. The processed log data is transmitted to the visualization interface for display and sent to the log analysis module. The flexible configuration and optimization of Logstash can effectively handle the processing requirements of a large amount of real-time log data, while ensuring the integrity, reliability, and security of the log data, and providing high-quality support for subsequent analysis and visualization modules.

C. Design of Log Analysis Module

As a crucial step in log analysis, log classification aims to categorize logs based on the semantic features of their content. This method constructs a classification system based on the differences in keywords among different types of logs, mainly covering three major categories: application logs, system logs, and security logs. Specifically, application logs mainly record business-related information such as user operations and abnormal events. System logs focus on monitoring the operating status of the operating system and hardware devices.

In terms of technical implementation, log classification is achieved through pre-processing and keyword matching. First, the system uses the Logstash component to extract the message of the logs according to preset rules. Then, it performs word segmentation and keyword-based feature matching. Finally, it completes the automatic classification of logs.

Log grading is to manage logs by dividing them into different levels according to their severity. Common log levels include ERROR, WARNING, INFO, and DEBUG. Each log

level represents different event importance and urgency: The ERROR level indicates a serious error that may lead to system or service interruption; the WARNING level indicates a potential problem. Although it does not affect the system operation, it requires further attention; the INFO level records routine events and reflects the normal state of the system; the DEBUG level is used for debugging information to help developers troubleshoot problems. The system can automatically assign logs to the corresponding levels through the level field in the logs and perform different processing according to the levels.

The specific implementation of log grading usually depends on the clearly identified level field in the logs. When generating logs, the log generators will identify the log levels through fields such as ERROR and INFO. The log system can extract these level information and adopt different processing strategies according to the severity of the logs. For serious errors, the system may trigger the alarm mechanism; for INFO-level logs, the system can archive them or use them for subsequent performance monitoring. Through this classification and grading mechanism, the system can manage and analyze a large amount of log data more efficiently.

Deep log analysis mainly relies on LLMs. Therefore, model selection is crucial for the performance of the log analysis system. Different LLMs have different capabilities and characteristics, and are particularly suitable for their respective NLP tasks. Selecting an appropriate LLMs for log analysis can significantly improve the efficiency and accuracy of the analysis task. In recent years, many high-performance language models have emerged both at home and abroad, such as BERT and GPT-3. However, these general models have obvious deficiencies when dealing with log analysis tasks. Although GPT-3 has a powerful generation ability, its scale of 175 billion parameters leads to extremely high computational resource requirements, and it lacks pertinence when processing structured log data. Although the BERT model is good at understanding context, its bidirectional Transformer architecture has low efficiency when processing sequential log data.

DeepSeek-R1-Distill-Qwen-7B was selected as the core model for this study, primarily due to its efficiency-oriented design. Through knowledge distillation from the Qwen foundation model with hundreds of billions of parameters, it retains the core capabilities of the base model while significantly reducing deployment requirements. The foundation model's pretraining data includes raw code corpora and technical documentation, enabling the distilled DeepSeek-R1-Distill-Qwen-7B to inherently handle structured formats and technical terminologies commonly found in log analysis. Compared to general-purpose LLMs, its medium-scale architecture demonstrates superior engineering applicability in production environments with high real-time demands.

Since fine-tuning this model requires sufficient computing power, this paper chooses to deploy and fine-tune the model on an RTX 3090 server. First, use the wget command on the server to download the source code of the model. Then, download the LLaMA-Factory fine-tuning assistant to simplify the fine-tuning process. Finally, configure the Python environment and conduct tests to ensure normal operation.

Loghub maintains a collection of system logs. Some of the logs are sourced from the generated data released in previous

studies, while the other part is collected from laboratory systems. The log data in Loghub covers large systems such as Apache and Windows, which is representative to a certain extent and has a sufficient number of samples. In this paper, the log data of Apache and Windows are selected for model fine-tuning to enhance the model's ability in log semantic understanding and analysis. And 1000 fine-tuning datasets are manually generated for each. The format of the fine-tuning dataset is shown in Fig. 4.

```
"instruction": "Please analyze the log:",
"input": "suEXEC mechanism enabled (wrapper: /usr/sbin/suexec)",
"output": "This log indicates that the suEXEC mechanism has been enabled. suEXEC
allows Apache to run CGI scripts with the privileges of a specific user."
```

Fig. 4. The format of fine-tuning dataset.

"Instruction" defines the specific operation to be performed on the log data. It serves as the guiding command for the analysis process. "Input" consists of the raw log messages to be processed, which may include system events, error reports, or performance metrics. This data typically comes from various sources like servers, applications, or network devices. "Output" represents the processed results generated after applying the instruction to the input. This could be identified anomalies, statistical summaries, visualized patterns, or actionable alerts that help administrators understand system behavior and troubleshoot issues.

This study employs LoRA technology for efficient fine-tuning, achieving model performance improvement through optimization of a minimal set of parameters. The LoRA fine-tuning process involves several critical hyperparameters, including the rank of low-rank matrices, scaling factors, learning rate, and training epochs. While the experiment adopts the default parameter configuration from LLaMA-Factory, specific adjustments were made to the number of training epochs. An insufficient number of epochs may lead to underfitting, failing to capture essential patterns in the data, whereas excessive epochs could induce overfitting, resulting in degraded performance on the validation set. Through systematic observation of model convergence trends and training loss patterns, we determined that 20 training epochs optimally balance stable training with effective task-specific learning. This configuration demonstrates synergistic effects with other hyperparameter settings, collectively contributing to enhanced model performance.

After the model training stage is completed, we have respectively selected the log data from four different types of systems, namely Apache, Windows, Linux, and HDFS, to conduct a comprehensive test on the model. Fig. 5 shows some typical examples of the test results. The empirical results clearly indicate that the default parameter configuration adopted by the model exhibits extremely good generalization ability. This means that the model can not only excellently adapt to the specific data used for training, but also effectively identify patterns, extract key information, and output accurate and reliable analysis results when facing log data from different systems.

```
log ( Apache ): suEXEC mechanism enabled (wrapper: /usr/sbin/suexec)
analysis result: This log indicates that the suEXEC mechanism has been
enabled. suEXEC allows Apache to run CGI scripts with the privileges of
a specific user.

log ( Windows ): Warning: Unrecognized packageExtended attribute.
analysis result: This log indicates that an unrecognized packageExtended
attribute has been encountered, which may be caused by the incompatibility
of the extended attributes of the file system.

log ( Linux ): ALERT exited abnormally with [1].
analysis result: This log indicates that the service or process has
experienced an abnormal exit, and the accompanying number [1] may be an
identifier for the error or exception.

log ( HDFS ): PacketResponder 1 for block blk_38865049064139660
terminating.
analysis result: This log indicates that the response of PacketResponder 1
to block blk_38865049064139660 has terminated, which shows that the
termination processing of this module has been completed.
```

Fig. 5. Examples of the test results.

D. Design of Data Storage Module

The data storage module is responsible for receiving processed results from the log analysis module and storing the analyzed data in a local MySQL database. MySQL, as an open-source relational database management system, is an ideal choice for log data storage due to its high performance, reliability, and ease of use. Utilizing structured query language, MySQL supports transaction processing, multi-user concurrent access, and complex data association operations, ensuring efficient storage and retrieval of log data. Its powerful indexing optimization mechanism significantly improves query speeds for massive log datasets, while ACID properties guarantee data integrity and consistency, preventing log loss or corruption. MySQL's flexible storage engine architecture allows optimization based on log processing requirements. Additionally, MySQL provides robust permission management and data encryption features to ensure the security of sensitive log information.

The database serves as the core data hub for the entire system, enabling structured log storage through normalized table schemas, which provides clean data sources for subsequent statistical analysis. Its efficient indexing mechanism allows the system to quickly respond to typical log queries, such as time-range searches and keyword retrievals. Meanwhile, transaction control and relational query capabilities maintain data consistency across different log modules. Furthermore, the data visualization module can directly retrieve standardized data from the database, eliminating redundant processing and optimizing overall operational efficiency. This centralized storage management model not only reduces system complexity but also lays a solid data foundation for future big data analytics, enhancing scalability and maintainability.

The log table is shown in Fig. 6, including various attributes of logs.

Log
Id: int
Time: timestamp
Message: text
Type: varchar (20)
Level: varchar (20)
Analysis_result: text

Fig. 6. The log table.

The user table is shown in Fig. 7, including user id, user name, and password.

User
Userid: int
Username: varchar (20)
Password: varchar (20)

Fig. 7. The user table.

E. Design of Data Visualization Module

The data visualization module serves as a critical component of the system, responsible for extracting processed log data from the database and presenting analytical results to users through various visual formats. Utilizing modern visualization technologies, this module enables dynamic and interactive data presentation, supporting user operations such as time-range filtering. By transforming abstract log data into intuitive charts and graphs, the visualization tool not only helps operations personnel quickly identify key information such as system operation trends and abnormal fluctuations, but also assists management in making data-driven decisions. This visual presentation approach significantly lowers the comprehension barrier for log data while enhancing the efficiency of system monitoring and troubleshooting.

The log data collected by Logstash is displayed line by line in the visualization interface, as shown in Fig. 8. Users can click on individual log entries to view their detailed information.

File does not exist: /var/www/html/awstats/awstats.pl
File does not exist: /var/www/html/cgi
File does not exist: /var/www/html/awstats.pl
File does not exist: /var/www/html/cgi
File does not exist: /var/www/html/awstats.pl
Directory index forbidden by rule: /var/www/html/

Fig. 8. The visualization of log collection.

The log classification results are presented to users in a pie chart format, as shown in Fig. 9. Users can click on each segment of the pie chart to view the quantity of log.

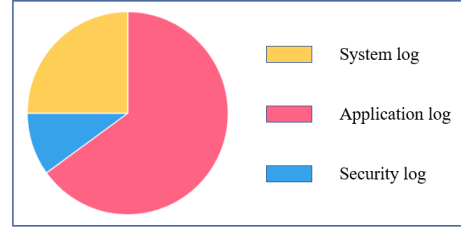


Fig. 9. The visualization of log classification.

IV. EVALUATION

A. Comparative Analysis

This paper compares the log analysis system based on LLMs with the ELK stack to highlight the advantages of the former. The analysis results are as follows:

1) *Technical adaptability analysis*: The traditional ELK stack relies heavily on rule-based log analysis, which requires predefined patterns and regular expressions to extract meaningful information from logs. While this approach works well for structured and predictable log formats, it struggles with variability in log structures, especially in dynamic environments where applications frequently update or generate heterogeneous logs. Any deviation from predefined rules leads to analysis failures, requiring constant manual updates to maintain accuracy. In contrast, the proposed log analysis system leverages fine-tuned LLMs to dynamically analyze log entries without rigid pattern matching. By training on diverse log datasets, the model develops an intrinsic understanding of log semantics, enabling it to generalize across different formats. This adaptability is particularly valuable in cloud-native and microservices architectures, where log formats vary significantly across services. Additionally, the model can infer contextual relationships between log entries, such as correlating error messages with system events, which traditional rule-based systems often miss. The result is a more scalable and resilient solution that reduces dependency on manual rule engineering while improving accuracy in anomaly detection.

2) *Comparison of cost-effectiveness*: The ELK stack, being open-source, offers low initial deployment costs, making it an attractive choice for organizations with limited budgets. However, its long-term operational expenses can be substantial due to the need for continuous rule maintenance. As log sources evolve, engineers must manually update parsing rules, grok patterns, and Elasticsearch mappings, which demands significant time and expertise. In large-scale deployments, this overhead escalates, particularly when dealing with custom applications or third-party services that frequently modify their log structures. The log analysis system based on LLMs, despite higher initial computational costs for model fine-tuning, drastically reduces ongoing operational expenses. Once trained, the model autonomously processes logs without frequent manual adjustments, minimizing human intervention. By deploying open-source LLMs locally,

organizations avoid recurring cloud-based API costs while retaining data privacy. Furthermore, the system's ability to automate complex tasks translates into reduced labor costs and faster incident resolution. Over time, the reduced reliance on specialized personnel for log management makes the LLM approach more cost-effective, especially for enterprises with large and dynamic IT infrastructures.

3) *Enhancement of analysis efficiency*: Rule-based systems like ELK excel at simple, deterministic tasks but lack the intelligence to analyze unstructured or ambiguous log data. They cannot infer intent, contextualize errors, or identify subtle patterns without explicit rules, limiting their usefulness in troubleshooting complex issues. For instance, a rule-based system might flag an error log but fail to correlate it with preceding warning messages or system metrics, leaving critical insights undiscovered. The system based LLMs overcomes these limitations by leveraging deep semantic understanding. It can analyze logs holistically, discerning relationships between seemingly unrelated entries, such as linking a memory leak in one service to cascading failures in another. The model's NLP capabilities allow it to summarize log trends, suggest probable causes, and even generate actionable recommendations. In performance benchmarking, the LLM approach demonstrates superior accuracy in anomaly detection and reduces false positives by interpreting logs contextually rather than relying on rigid rules. This leads to faster mean time to resolution, particularly in environments where logs are voluminous and diverse.

B. Shortcoming

1) *Performance challenges*: Although the system employs optimization techniques, its real-time log analysis capabilities remain insufficient when handling high-velocity, large-scale data streams. As data volume grows, latency in log ingestion, processing, and response times increases, impacting the system's ability to provide timely insights. Additionally, the current infrastructure may struggle with peak loads, leading to potential bottlenecks in log aggregation and storage. Further optimizations, such as stream processing enhancements and dynamic resource allocation, are needed to improve scalability and real-time performance.

2) *Model dependence and limitations*: The current log analysis model, while optimized for efficiency, has inherent constraints due to its relatively small parameter size. Although lightweight models reduce computational costs, they may lack the depth to accurately detect complex log patterns, rare anomalies, or sophisticated attack signatures. This limitation increases the risk of false negatives, where critical issues go undetected. To address this, integrating larger pre-trained models or adopting ensemble methods could enhance detection accuracy while balancing resource consumption.

3) *Potential problems of architecture scalability*: As business requirements evolve and system complexity increases, the current architecture may face challenges in maintaining seamless coordination among modules. For instance, the log collection, storage, and analysis components might experience integration bottlenecks, leading to inefficiencies in data flow. Additionally, horizontal scaling could introduce synchronization and consistency challenges.

A more modular, microservices-based design with well-defined APIs and event-driven communication could improve flexibility, allowing the system to adapt dynamically to changing workloads and business needs.

V. CONCLUSION

As a pivotal technology for enterprise security operations and compliance management, log analysis systems have garnered significant attention in both research and application. This study conducts a systematic review of current log analysis technologies and proposes an intelligent log analysis system based on LLMs. The system employs Logstash for log collection and preprocessing, integrating keyword matching and severity-level identification to achieve automated log classification and grading. A key innovation lies in the incorporation of DeepSeek-R1-Distill-Qwen-7B model, enhanced with LoRA fine-tuning techniques, which significantly improves analytical effect.

While the current system demonstrates practical applicability, further advancements remain possible. Future research can explore multi-model fusion strategies to leverage complementary strengths across different architectures, while also investigating more efficient fine-tuning methods to reduce computational overhead. These efforts will drive innovation and elevate the state-of-the-art in log analysis.

ACKNOWLEDGMENT

This work was supported by National Key Research and Development Program of China (No. 2023YFB3106400, 2023QY1202), the National Natural Science Foundation of China (U2336203, U1836210), the Key Research and Development Science and Technology of Hainan Province (GHYF2022010), and Beijing Natural Science Foundation (L242015, 4242031).

REFERENCES

- [1] A. Oliner, A. Ganapathi, and W. Xu, "Advances and challenges in log analysis," *Commun. ACM*, vol. 55, no. 2, pp. 55–61, 2012.
- [2] P. P. Bavaskar, O. Kemker, and A. K. Sinha, "A Survey On: Log Analysis With Elk Stack Tool," *IJRAR-Int. J. Res. Anal. Rev.*, vol. 6, no. 4, pp. 965–968, 2019.
- [3] Y. Wang, "Design of Visual Log Analysis System," in 2023 IEEE International Conference on Sensors, Electronics and Computer Engineering (ICSECE), JinZhou, China, Aug. 2023, pp. 1649–1652.
- [4] Y. Sun, S. Guo, and Z. Chen, "Intelligent log analysis system for massive and multi-source security logs: Mmslas design and implementation plan," in 2019 15th International Conference on Mobile Ad-Hoc and Sensor Networks (MSN), Shenzhen, China, Dec. 2019, pp. 416–421.
- [5] K. Subramanian and K. Subramanian, "Introducing the Splunk platform," *Practical Splunk Search Processing Language: A Guide for Mastering SPL Commands for Maximum Efficiency and Outcome*, pp. 1–38, 2020.
- [6] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security (CCS)*, Dallas, TX, USA, Oct. 2017, pp. 1285–1298.
- [7] Hochreiter S, Schmidhuber J. Long short-term memory[J]. *Neural computation*, 1997, 9(8): 1735–1780.
- [8] Zaremba W, Sutskever I, Vinyals O. Recurrent neural network regularization[J]. *arXiv preprint arXiv:1409.2329*, 2014.
- [9] E. J. Hu et al., "Lora: Low-rank adaptation of large language models.," *ICLR*, vol. 1, no. 2, p. 3, 2022.

- [10] Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1. 2019: 4171-4186.